Financial Time Series Forecasting – Stock Market Prediction

Sohen Patel

Agenda

- Review of Tatsat's Case Study
- Extending Tatsat's Work
  - Introducing additional features to dataset
  - Applying XGBoost and SVM models
  - Feature Selection using SelectKBest
  - Model Selection
  - Hyperparameter Tuning
  - Trading Strategy Implementation
  - Shap Values
  - White Reality Check
  - Monte Carlo Simulations
- Conclusion

Review of Tatsat’s Case Study

Problem Definition & Data Scope

- Tatsat's *Chapter 5 Case Study 1: Stock Price Prediction* focuses on **predicting the weekly return of Microsoft (MSFT)** stock using historical and correlated asset data through the application of various ML models
- Data for the case study is sourced from Yahoo Finance and the FRED Website
- This case study specifically uses correlated assets as predictors: I*BM, Alphabet (GOOGL), USD/JPY, GBP/USD, S&P 500, Dow Jones*, and *VIX*
- By lagging Microsoft's own historical returns across 5, 15, 30, and 60-day windows, we force the models to evaluate the momentum and impact of broader market changes to MSFT returns

Data Preparation

Predictors

Lagged 5-day log returns	
Stocks	IBM
	GOOG
Currency	USD/JPY
	GBP/USD
Market Indices	S&P 500
	Dow Jones
	VIX
MSFT lagged historical returns	5-day lagged return
	15-day lagged return
	30-day lagged return
	60-day lagged return

Target Variable (Y)

Future 5-day log return of MSFT

- The dataset contains 176 distinct 5-day periods between 2010 and 2020. Additionally the dataset contains no NaNs

```
return_period = 5
Y = np.log(stk_data.loc[:, ('Adj Close', 'MSFT')]).diff(return_period).\
shift(-return_period)
Y.name = Y.name[-1]+'_pred'

X1 = np.log(stk_data.loc[:, ('Adj Close', ('GOOGL', 'IBM'))]).diff(return_period)
X1.columns = X1.columns.droplevel()
X2 = np.log(ccy_data).diff(return_period)
X3 = np.log(idx_data).diff(return_period)

X4 = pd.concat([np.log(stk_data.loc[:, ('Adj Close', 'MSFT')]).diff(i) \

for i in [return_period, return_period*3,\
return_period*6, return_period*12]], axis=1).dropna()
X4.columns = ['MSFT_DT', 'MSFT_3DT', 'MSFT_6DT', 'MSFT_12DT']

X = pd.concat([X1, X2, X3, X4], axis=1)

dataset = pd.concat([Y, X], axis=1).dropna().iloc[::return_period, :]
Y = dataset.loc[:, Y.name]
X = dataset.loc[:, X.columns]
```

```
print("Dataset size (rows, columns):", dataset.shape)
print("\nNumber of NaN values per column:\n", dataset.isnull().sum())
```

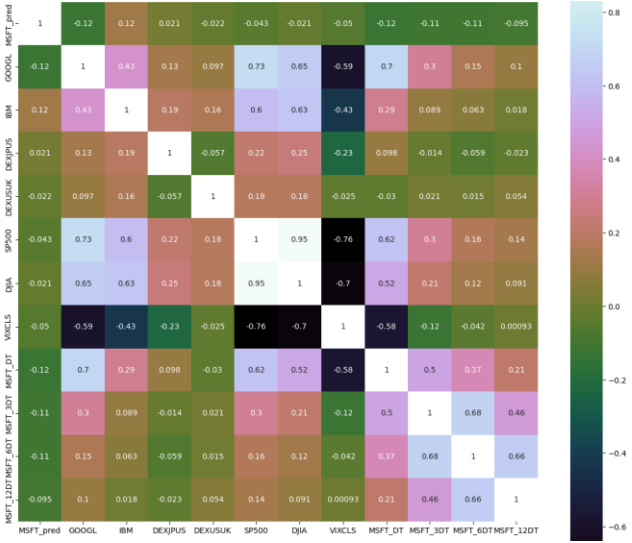
```
Dataset size (rows, columns): (176, 12)

Number of NaN values per column:
 MSFT_pred    0
GOOGL         0
IBM           0
DEXJPUS       0
DEXUSUK       0
SP500         0
DJIA          0
VIXCLS        0
MSFT_DT       0
MSFT_3DT      0
MSFT_6DT      0
MSFT_12DT     0
dtype: int64
```

Exploratory Data Analysis (EDA)

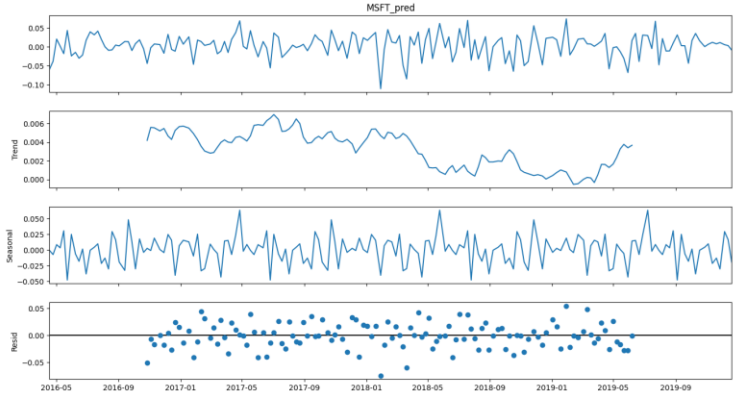
Correlation Matrix

- We can observe a correlation between the predicted variable with the lagged returns of MSFT (5, 15, 30, and 60 days)
- VIX is negatively correlated with all equity indices (−0.77 with S&P 500, −0.74 with DJIA), which is expected as volatility decreases as markets rise



Time Series Analysis

- MSFT shows a general upward trend in weekly returns, driven by the stock's strong price run-up in recent years
- The upward trend is expected to appear as a positive constant/bias term in our models
- The residual (white noise) term is small across the entire series



Feature Selection

Approach

- SelectKBest with *f_regression* was used to score and rank all features by their linear predictive power over MSFT returns

Best Features

- **MSFT_DT** and **GOOGL** are the strongest predictors
- **IBM** and MSFT's own lagged returns at 15 and 30 days also rank highly
- Microsoft's own price history consistently appears among the most important features

Worse Features

- VIX, S&P 500, DEXUSUK, and DJIA score near zero

```python
bestfeatures = SelectKBest(k=5, score_func=f_regression)
fit = bestfeatures.fit(X,Y)
dfscores = pd.DataFrame(fit.scores_)
dfcolumns = pd.DataFrame(X.columns)
#concat two dataframes for better visualization
featureScores = pd.concat([dfcolumns,dfscores],axis=1)
featureScores.columns = ['Specs','Score']  #naming the dataframe columns
featureScores.nlargest(10,'Score').set_index('Specs')  #print 10 best features
```

Specs	Score
MSFT_DT	2.651
GOOGL	2.500
IBM	2.427
MSFT_3DT	2.194
MSFT_6DT	2.116
MSFT_12DT	1.569
VIXCLS	0.439
SP500	0.327
DEXUSUK	0.082
DJIA	0.079

Experimental Setup & Evaluation

Data Split

- 80% of the dataset is used for training, 20% for testing
- The split is chronological to preserve the order of observations in the time series

Model Evaluation

- Models are evaluated using 10-fold cross validation on the training set to account for randomness
  - **Note:** KFold analysis is not run on time-series models
- Models are scored based on their Mean Squared Error (MSE)

Models Compared

- 11 supervised regression models are tested: Linear Regression, Lasso, Elastic Net, KNN, CART, SVR, MLP, AdaBoost, Gradient Boosting, Random Forest, and Extra Trees
- ARIMA and LSTM are also included as time-series models

```
names = []
kfold_results = []
test_results = []
train_results = []
for name, model in models:
    names.append(name)

    ## K Fold analysis:

    kfold = KFold(n_splits=num_folds) # Removed random_state as shuffle is
    False by default for KFold
    #converted mean square error to positive. The lower the beter
    cv_results = -1* cross_val_score(model, X_train, Y_train, cv=kfold,
    scoring=scoring)
    kfold_results.append(cv_results)


    # Full Training period
    res = model.fit(X_train, Y_train)
    train_result = mean_squared_error(res.predict(X_train), Y_train)
    train_results.append(train_result)

    # Test results
    test_result = mean_squared_error(res.predict(X_test), Y_test)
    test_results.append(test_result)

    msg = "%s: %f (%f) %f %f" % (name, cv_results.mean(), cv_results.std(),
    train_result, test_result)
    print(msg)
```

Regression and Tree Regression algorithms

```
models = []
models.append(('LR', LinearRegression()))
models.append(('LASSO', Lasso()))
models.append(('EN', ElasticNet()))
models.append(('KNN', KNeighborsRegressor()))
models.append(('CART', DecisionTreeRegressor()))
models.append(('SVR', SVR()))
```

Neural Network algorithms

```
models.append(('MLP', MLPRegressor()))
```

Ensable Models

```
# Boosting methods
models.append(('ABR', AdaBoostRegressor()))
models.append(('GBR', GradientBoostingRegressor()))
# Bagging methods
models.append(('RFR', RandomForestRegressor()))
models.append(('ETR', ExtraTreesRegressor()))
```

Baseline Machine Learning Models

Regression Models

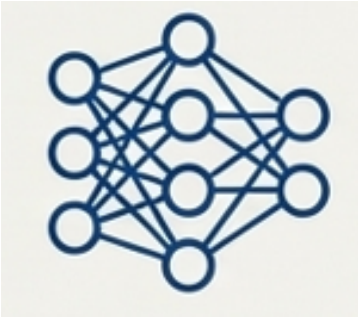
- Linear Regression (LR)
- LASSO
- Elastic Net (EN)
- KNN
- SVR

Ensembles & Trees

- CART
- RFR
- GBR
- ETR
- ABR

Deep ML

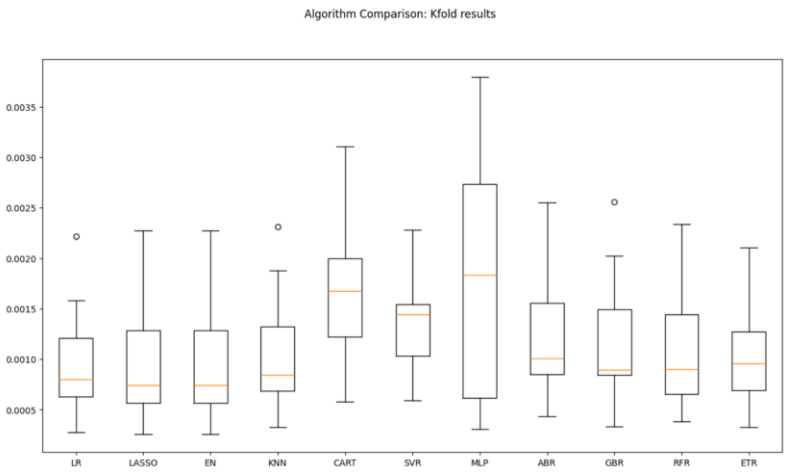
- MLP

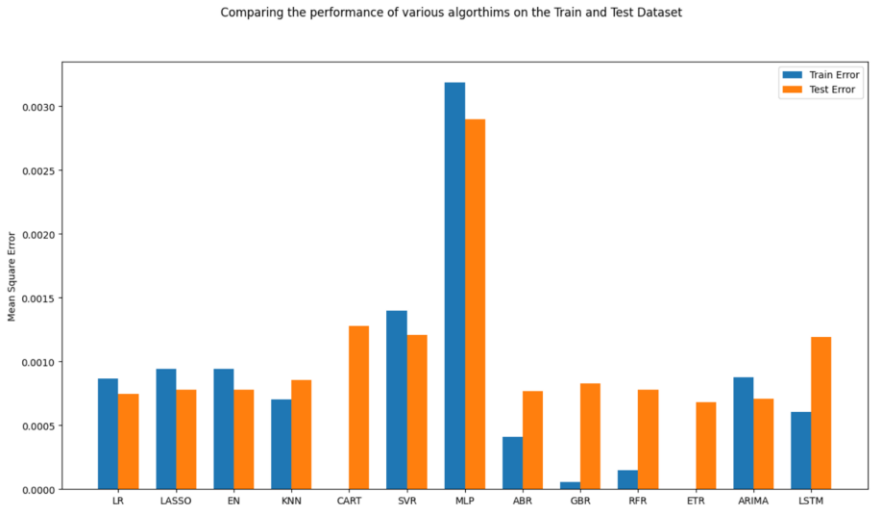


Statistical Time

- ARIMA
- Long Short-Ter
  (LSTM)

Results





- LR, Lasso, and EN perform best overall, confirming a strong linear relationship in the data
- Ensemble methods (such as GBR and RFR) performed well on the training set but poorly on the test set, indicating overfitting
- MLP underperforms on both sets, likely due to insufficient data for a neural network
- ARIMA achieves comparable performance to the linear models and outperforms LSTM on the test set

Time Series Modeling (ARIMA)

Setup

- MSFT lagged returns are removed as inputs and only the correlated assets are used as exogenous variables
- Exogenous variables: GOOGL, IBM, DEXJPUS, SP500, DJIA, VIXCLS

Grid Search

- p, d, and q values are each iterated across a defined range to find the lowest training MSE
- Best configuration: **ARIMA(2,0,1)**

Results

- *Training MSE:* 0.00085
- *Test MSE:* 0.00071

```
X_train_ARIMA=X_train.loc[:, ['GOOGL', 'IBM', 'DEXJPUS', 'SP500', 'DJIA',
'VIXCLS']]
X_test_ARIMA=X_test.loc[:, ['GOOGL', 'IBM', 'DEXJPUS', 'SP500', 'DJIA',
'VIXCLS']]
tr_len = len(X_train_ARIMA)
te_len = len(X_test_ARIMA)
to_len = len (X)
```

```
# evaluate combinations of p, d and q values for an ARIMA model
def evaluate_models(p_values, d_values, q_values):
    best_score, best_cfg = float("inf"), None
    for p in p_values:
        for d in d_values:
            for q in q_values:
                order = (p,d,q)
                try:
                    mse = evaluate_arima_model(order)
                    if mse < best_score:
                        best_score, best_cfg = mse, order
                    print('ARIMA%s MSE=%.7f' % (order,mse))
                except:
                    continue
    print('Best ARIMA%s MSE=%.7f' % (best_cfg, best_score))
```

Overall Comparison

Why ARIMA performed well in predicting MSFT returns

1. The EDA confirmed a **linear, autocorrelated structure in MSFT weekly returns**. ARIMA is built for this type of data, making it a better fit compared to complex models that assume non-linearity

2. Ensemble methods like CART, ABR, and GBR all showed very **low training error but high test error**. ARIMA avoids this by not overfitting to short-term noise in the training data

3. ARIMA performs comparably to the best linear regression models (LR, Lasso, EN) while also **capturing the time series structure of the data** through its AR and MA components

Extending Tatsat’s Work

Introducing additional features to dataset

Ten new features were added to Tatsat's original feature set. The table below summarizes the **newly added 5 price-based features**. The price-based features are all derived from MSFT's own historical price data *

Price-based		
	Feature Name	**Description**
1	RSI (14-day) *Relative Strength Index*	A 0–100 score showing whether MSFT has been rising or falling too quickly. Values above 70 suggest overbought conditions, while below 30 suggest oversold, helping anticipate short-term reversals.
2	MACD *Moving Average Convergence Divergence*	Measures the gap between short-term and long-term trends using exponential moving averages. When it crosses above zero, it signals bullish momentum; below zero signals bearish momentum.
3	Bollinger Band Width	Measures how wide the price bands are over a 20-day window. Narrow bands indicate low volatility and often precede large price movements or breakouts.
4	ATR (14-day) *Average True Range*	Measures the average size of daily price movements. A higher ATR reflects increased volatility, uncertainty, or stronger institutional trading activity.
5	ROC (10-day) *Rate of Change*	Measures the percentage change in price over the past 10 days. It acts as a direct momentum signal, showing whether the stock has been consistently rising or falling.

* MSFT historical price data sourced from Yahoo Finance via yfinance

Introducing additional features to dataset

The table below summarizes the **newly added 5 non-price based features.** The non price-based features allow the model to incorporate broader market conditions beyond MSFT's own price movements *

Non-Price based		
	Feature Name	**Description**
6	Gold Return	Measures the 5-day log return of gold. Since gold is a safe-haven asset, rising gold often indicates investors are moving out of equities, which can be bearish for MSFT.
7	Crude Oil Return	Measures the 5-day log return of oil. Rising oil prices signal inflationary pressure, which increases discount rates and negatively impacts growth stock valuations like MSFT.
8	10-Year Treasury Yield Change	Weekly change in the US 10-year yield. Higher yields raise the discount rate applied to MSFT's future cash flows, reducing today's fair value.
9	Bitcoin Return	Measures the 5-day log return of Bitcoin. Bitcoin acts as a proxy for risk appetite, where rising BTC suggests stronger demand for risky and growth-oriented assets.
10	Rolling Beta (52-day)	Measures how sensitive MSFT is to movements in the S&P 500 over a 52-day rolling window. A rising beta indicates MSFT is becoming more volatile relative to the market and amplifying market movements.

* Data sourced from Yahoo Finance (yfinance) for Gold (GC=F), Crude Oil (CL=F), and Bitcoin (BTC-USD) returns; US 10-Year Treasury Yield from FRED (DGS10); Rolling Beta computed from MSFT and S&P 500 (^GSPC) price series

Applying XGBoost and SVM models

XGBoost and SVM models were added to the original set of models implemented by Tatsat

XGBoost

- XGBoost is used to model non-linear relationships in stock price data by combining multiple decision trees to improve prediction accuracy
- XGBoost Builds an ensemble of decision trees one at a time, where each new tree is trained to correct the errors left by the previous ones. It uses gradient descent to minimise the loss function at each step

Train MSE: 0.00000029
Test MSE: 0.00210788

- XGBoost performed poorly by severely overfitting. Since stock returns are noisy and largely linear, this leaves little non-linear structure for tree-based models to exploit

SVM

- SVR was added as it is a robust regression model that, unlike linear models, can capture more complex relationships by finding the best fit within a defined tolerance band rather than minimising total prediction error directly
- SVM is sensitive to the scale of input features. StandardScaler was applied first to normalise all features to mean 0 and standard deviation 1

Train MSE: 0.00245631
Test MSE: 0.00254190

- SVR avoided overfitting by maintaining consistent train and test MSE, however it underperformed linear models such as LASSO and EN overall

Feature Selection using SelectKBest

- Feature selection was performed using **SelectKBest** with *f_regression* to rank predictors by their linear relationship with the target
  - *f_regression* applies linear regression between each feature and the target, evaluating the strength of the relationship using the F-statistic. A higher F-value indicates greater predictive power [1]
  - k='all' was used to rank all features before manually removing the lowest-scoring one.
- The 5-day log return of **USD/JPY (DEXJPUS) was ranked as the lowest-scoring feature** and was **removed**
  - DEXJPUS likely had a low score because its information is largely captured by the other currency feature already in the dataset

```python
from sklearn.feature_selection import SelectKBest, f_regression
import pandas as pd

selector = SelectKBest(score_func=f_regression, k='all')
selector.fit(X, Y)

feature_scores = pd.DataFrame({
    'Feature': X.columns,
    'Score': selector.scores_
}).sort_values('Score', ascending=False).reset_index(drop=True)

print(feature_scores.to_string(index=False))
```

Feature	Score
ROC	7.227
MACD	6.009
MSFT_3DT	3.783
Oil_Return	3.236
MSFT_12DT	3.058
MSFT_DT	2.767
RSI	2.255
BTC_Return	2.112
MSFT_6DT	1.687
DJIA	0.841
SP500	0.802
ATR	0.783
Rolling_Beta	0.594
GOOGL	0.575
IBM	0.508
Treasury_Change	0.082
DEXUSUK	0.054
BB_Width	0.023
VIXCLS	0.009
Gold_Return	0.002
DEXJPUS	0.000

```python
worst_feature = feature_scores.iloc[-1, 0]
print("Dropping worst feature:", worst_feature)

X = X.drop(columns=[worst_feature])

print("Total features after selection:", X.shape[1])
print("Remaining features:", list(X.columns))
```

```
Dropping worst feature: DEXJPUS
```


https://medium.com/@mathematicaldatascience/feature-selection-using-selectkbest-a-complete-guide-for-data-science-professionals-3a94ede2a796

Model Selection

Model	Train MSE	Test MSE
ARIMA	0.00104340	0.00088553
LASSO	0.00107182	0.00088892
EN	0.00107182	0.00088892
KNN	0.00089487	0.00103464
LSTM	0.00063319	0.00114488
LR	0.00092372	0.00114951
ETR	0.00000000	0.00117514
RFR	0.00016779	0.00141766
ABR	0.00043302	0.00142176
GBR	0.00004048	0.00168755
XGB	0.00000029	0.00210788
SVR	0.00245631	0.00254190
CART	0.00000000	0.00395056
MLP	0.00746785	0.04729039

- **ARIMA** achieved the **lowest test MSE** and was selected as the best-performing model
- ARIMA's strong performance is explained by its ability to **model the autocorrelated nature of weekly stock returns**, capturing temporal patterns that sklearn models treat independently
- Among ML models, LASSO and EN tied for best performance, suggesting that linear relationships capture a meaningful portion of the signal
- Tree-based ensemble models (XGB, GBR, RFR, ETR) and CART all showed severe overfitting, with near-zero training MSE but significantly higher test MSE
- LSTM underperformed despite being a deep learning time series model, likely due to the small dataset size limiting its ability to learn meaningful temporal patterns

ARIMA was selected for hyperparameter tuning and for implementing the trading strategy

ARIMA - Input Features

ARIMA(2,0,1) was re-evaluated after extending the exogenous feature set with the 5 newly added exogenous market variables, to assess whether the additional features improved out-of-sample performance

Original Features

```
# evaluate an ARIMA model for a given order (p,d,q)

X_train_ARIMA = X_train.loc[:, ['GOOGL', 'IBM', 'DJIA', 'VIXCLS', 'SP500']]
X_test_ARIMA  = X_test.loc[:, ['GOOGL', 'IBM', 'DJIA', 'VIXCLS', 'SP500']]
```

```
ARIMA(1,0,0) test MSE : 0.00137068
ARIMA(2,0,1) test MSE : 0.00087627
```

Additional Features

```
# evaluate an ARIMA model for a given order (p,d,q)

ARIMA_FEATURES = [
    'GOOGL', 'IBM', 'DJIA', 'VIXCLS', 'SP500',          # Original Tatsat
    features
    'Gold_Return', 'Oil_Return', 'Treasury_Change',        # New exogenous
    features
    'BTC_Return', 'Rolling_Beta'
]
```

```
ARIMA(1,0,0) test MSE : 0.00137068
ARIMA(2,0,1) test MSE : 0.00097253
```

Takeaway: Adding the 5 new exogenous features slightly worsened ARIMA(2,0,1)'s performance, with test MSE increasing from 0.00087 to 0.00097. This suggests the new features do not add meaningful predictive signal beyond what the original features already capture. As a result, the original Tatsat feature set was retained for the trading strategy

Hyperparameter Tuning

```python
# evaluate combinations of p, d and q values for an ARIMA model
def evaluate_models(p_values, d_values, q_values):
    best_score, best_cfg = float("inf"), None
    for p in p_values:
        for d in d_values:
            for q in q_values:
                order = (p,d,q)
                try:
                    mse = evaluate_arima_model(order)
                    if mse < best_score:
                        best_score, best_cfg = mse, order
                    print('ARIMA%s MSE=%.7f' % (order,mse))
                except:
                    continue
    print('Best ARIMA%s MSE=%.7f' % (best_cfg, best_score))

# evaluate parameters
p_values = [0, 1, 2]
d_values = range(0, 2)
q_values = range(0, 2)
warnings.filterwarnings("ignore")
evaluate_models(p_values, d_values, q_values)
```

```
ARIMA(0, 0, 0) MSE=0.0010539
ARIMA(0, 0, 1) MSE=0.0010304
ARIMA(0, 1, 0) MSE=0.0020785
ARIMA(0, 1, 1) MSE=0.0011079
ARIMA(1, 0, 0) MSE=0.0010434
ARIMA(1, 0, 1) MSE=0.0010217
ARIMA(1, 1, 0) MSE=0.0017638
ARIMA(1, 1, 1) MSE=0.0011123
ARIMA(2, 0, 0) MSE=0.0010077
ARIMA(2, 0, 1) MSE=0.0010060
ARIMA(2, 1, 0) MSE=0.0014240
ARIMA(2, 1, 1) MSE=0.0011172
Best ARIMA(2, 0, 1) MSE=0.0010060
```

- ARIMA models are defined by three parameters: **(p, d, q)**
  - **p** = number of past return values used
  - **d** = number of times the series is differenced
  - **q** = number of past forecast errors used
- A **grid search** was performed by testing multiple combinations of:
  - **p = 0, 1, 2**
  - **d = 0, 1**
  - **q = 0, 1**
- For each combination, the model was fit on the training set and evaluated using **mean squared error (MSE)**
- Based on the grid search, **ARIMA(2,0,1)** was identified as the optimal model
- ARIMA(2,0,1) achieved a lower test MSE than the baseline ARIMA(1,0,0), confirming improved generalization

Trading Strategy Implementation - Background

Each week, ARIMA(2,0,1) **predicts MSFT's 5-day return**. The prediction is compared against a **signal threshold** to decide the **position**

Position Types

Long (signal = +1)	Flat (signal = 0)	Short (signal = -1)
Prediction above threshold. Buy MSFT for the week	Prediction too close to zero. No trade is done, cash is held	Predicted return is below the negative threshold, sell MSFT short for the week

Signal Threshold

The threshold is set at the 40th percentile of absolute training predictions. Only signals stronger than this threshold trigger a trade, weaker signals result in no position. This prevents the strategy from overtrading on noisy, low-confidence predictions.

```python
# The 40th percentile of the absolute train predictions is used as the threshold
threshold = np.percentile(np.abs(y_pred_train_arima.values), 40)
print(f"\nsignal threshold (40th percentile of |train predictions|):
{threshold:.6f}")

def make_positions(predictions, threshold):
    # Convert the raw predicted return into a discrete trading signal
    # long if prediction is above the threshold
    # short if prediction is below the negative threshold
    # flat if the signal is not strong enough to act on
    predictions = np.asarray(predictions)
    return np.where(
        predictions >  threshold,  1,
        np.where(predictions < -threshold, -1, 0)
    )

positions_train = make_positions(y_pred_train_arima.values, threshold)
positions_test  = make_positions(y_pred_test_arima.values,  threshold)
```

Performance Metrics Used

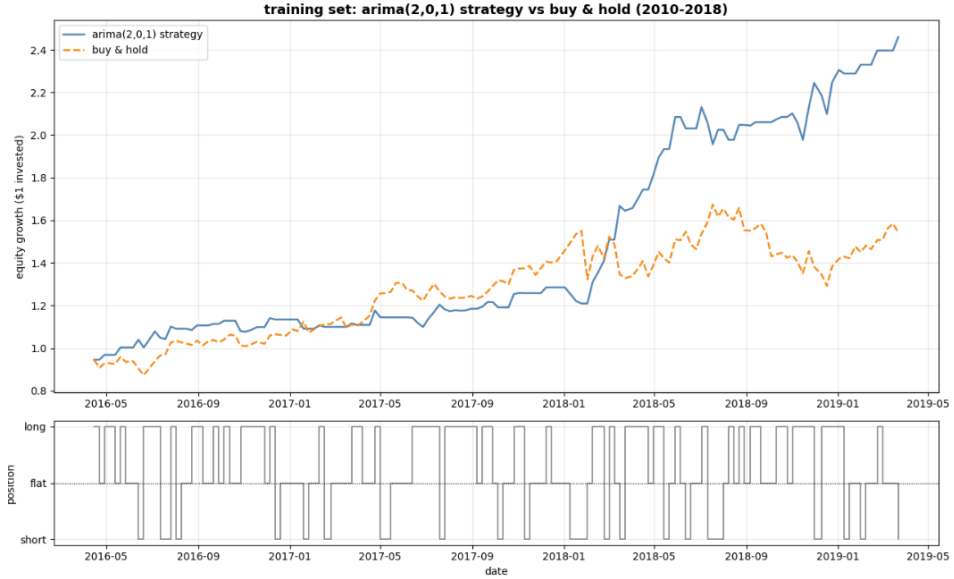
Metric	Interpretation
Sharpe Ratio	Measures risk-adjusted return by comparing average returns to volatility; higher values indicate better returns per unit of risk
CAGR	Represents the average annual growth rate of an investment; higher values indicate stronger long-term performance
Profit Factor	Measures trading performance as total profit divided by total losses; values above 1 indicate a profitable strategy

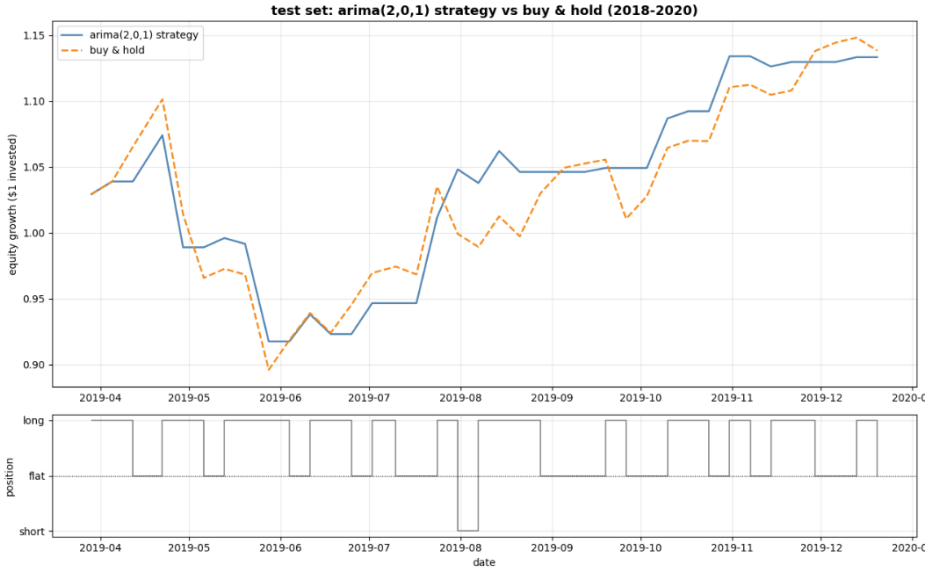
```python
def sharpe_ratio(r, ppy=PERIODS_PER_YEAR):
    # Annualised sharpe ratio with no risk free rate adjustment
    r = np.asarray(r)
    std = np.std(r, ddof=0)
    return 0.0 if std == 0 else np.sqrt(ppy) * np.mean(r) / std

def cagr(r, ppy=PERIODS_PER_YEAR):
    # Compound annual growth rate
    # guarding against negative compounded growth to avoid a complex number
    result
    r = np.asarray(r)
    growth = np.prod(1 + r)
    if growth <= 0 or len(r) == 0:
        return np.nan
    return growth ** (ppy / len(r)) - 1

def profit_factor(r):
    # Gross profit divided by gross loss on active trades only
    # flat periods are excluded so they do not distort the ratio
    r = np.asarray(r)
    active = r[r != 0]
    gross_profit = active[active > 0].sum()
    gross_loss   = np.abs(active[active < 0].sum())
    return np.nan if gross_loss == 0 else gross_profit / gross_loss
```

Trading Strategy Implementation - Equity Curves






```
=====================================
training set
=====================================
arima(2,0,1) strategy
-------------------------------------
  sharpe ratio  : 1.7872
  cagr          : 35.86%
  profit factor : 2.3609
  win rate      : 58.43%
  total return  : 145.95%

buy & hold
-------------------------------------
  sharpe ratio  : 0.7608
  cagr          : 15.94%
  profit factor : 1.3352
  win rate      : 57.43%
  total return  : 54.41%
```

```
=====================================
test set
=====================================
arima(2,0,1) strategy
-------------------------------------
  sharpe ratio  : 1.0107
  cagr          : 18.05%
  profit factor : 1.6689
  win rate      : 68.18%
  total return  : 13.33%

buy & hold
-------------------------------------
  sharpe ratio  : 0.9288
  cagr          : 18.75%
  profit factor : 1.4237
  win rate      : 65.79%
  total return  : 13.83%
```

Trading Strategy Implementation - Results

Test Data Results			
	ARIMA Strategy	Buy & Hold	Interpretation
Sharpe Ratio	1.0107	0.9288	ARIMA generates more return per unit of risk
CAGR	18.05%	18.75%	Similar annual growth rate over the test period
Profit Factor	1.6689	1.4237	ARIMA earns $1.67 for every $1 lost vs $1.42 for buy & hold

Training Data Results

- The ARIMA strategy significantly outperformed buy and hold on the training set, achieving nearly triple the total return. This suggests that the model captures directional patterns in the training period

Test Data Results

- The Sharpe ratio of 1.01 vs 0.93 and profit factor of 1.67 vs 1.42 both confirm **ARIMA generates better returns per unit of risk** compared to passively holding MSFT
- Both strategies produced similar raw returns, the ARIMA strategy marginally trailed buy and hold, but achieved this with better risk control
- ARIMA avoided the mid-2019 drawdown by going flat during uncertainty, delivering competitive returns with lower volatility than buy and hold

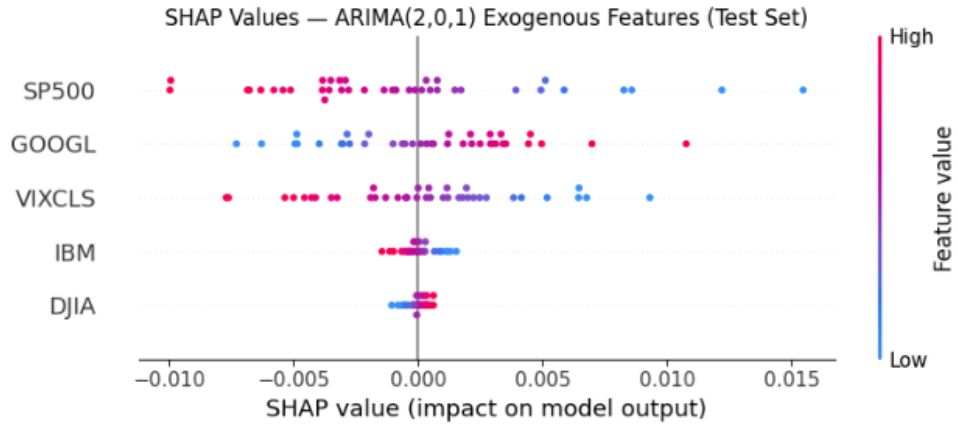
Shap Values

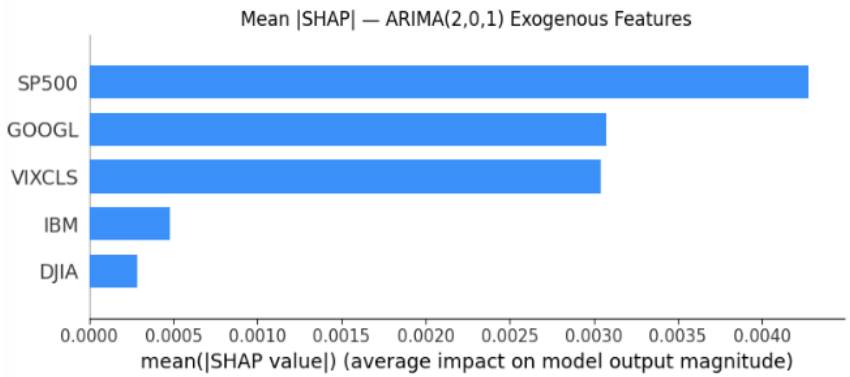
SHAP values were computed on the exogenous features of the final ARIMA(2,0,1) model using the linear SHAP method (SHAP value = coefficient x centred feature value).

Feature	Coefficient	Mean |SHAP| (test)
SP500	-0.357588	0.004276
GOOGL	+0.166795	0.003073
VIXCLS	-0.027214	0.003038
IBM	-0.024123	0.000478
DJIA	+0.023584	0.000285

- SP500 is the most influential feature based on mean |SHAP| values, reflecting strong market-wide effects
- GOOGL's positive coefficient alongside SP500's negative sign reflects multicollinearity between correlated indices
- VIXCLS shows a negative relationship, consistent with higher volatility predicting lower returns
- IBM and DJIA have low feature importance, contributing minimally to model predictions

Shap Values





Feature	Coefficient	Mean |SHAP| (test)
SP500	-0.357588	0.004276
GOOGL	0.166795	0.003073
VIXCLS	-0.027214	0.003038
IBM	-0.024123	0.000478
DJIA	0.023584	0.000285

White Reality Check

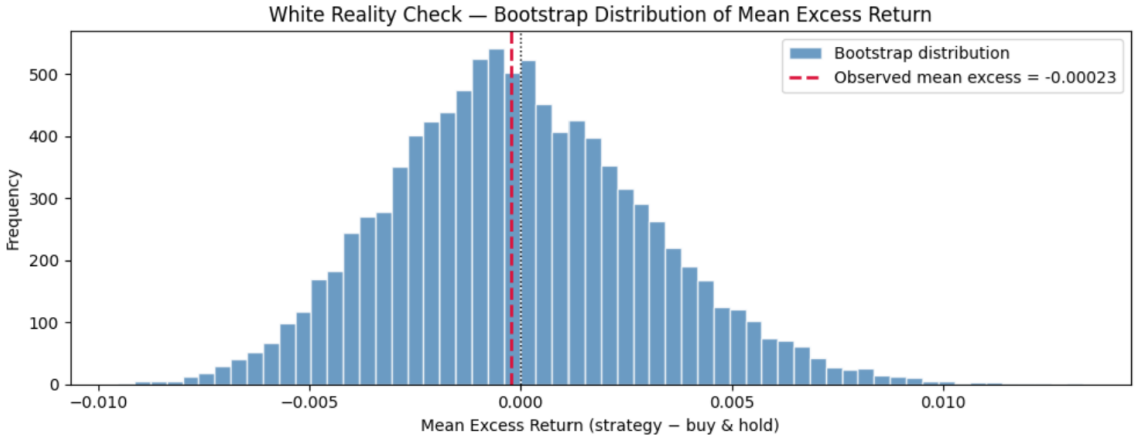
Tests whether the ARIMA(2,0,1) strategy generates statistically significant excess returns over the buy & hold benchmark on the test set, accounting for data snooping.

- **H_0:** Strategy does NOT outperform buy & hold
- **Method:** Bootstrap resampling (10,000 replications) of mean excess return under H_0

Result	Value
Mean Excess Return	−0.000229 per period
Test Statistic (Z)	−0.0739
Bootstrap p-value	0.5021
Reject H_0 at 5%?	No

Interpretation: The strategy underperforms buy & hold by ~0.02% per period on the test set and the p-value of 0.5021 is far above the 5% threshold. The ARIMA strategy does not generate statistically significant excess returns over a passive benchmark in the 2018–2019 test window.

White Reality Check




```
==================================================
  White Reality Check — ARIMA(2,0,1) vs Buy & Hold
  (Test Set)
==================================================
  Mean Excess Return : -0.000229
  Test Statistic (Z) : -0.0739
  Bootstrap p-value  : 0.5021
  Reject H0 at 5%?   : NO
==================================================
```

Monte Carlo Permutation Test

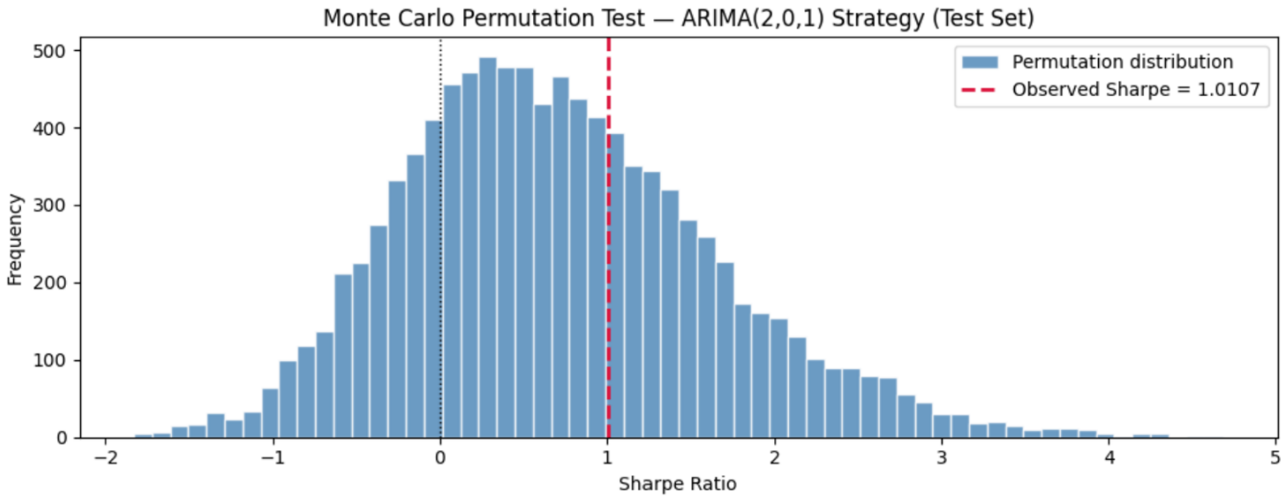
Tests whether the strategy's Sharpe ratio is better than what a randomly timed strategy would achieve by chance, using the same actual returns but shuffled signal positions.

- **H_0**: Strategy performance is no better than random signal timing
- **Method:** 10,000 permutations of actual returns with shuffled positions

Result	Value
Observed Sharpe Ratio	+1.0107
Permutation p-value	0.3433
Reject H_0 at 5%?	No

Interpretation: With a p-value of 0.343, the null hypothesis cannot be rejected. This indicates the strategy's Sharpe ratio is not statistically distinguishable from random timing

Monte Carlo Permutation Test




```
==================================================
  Monte Carlo Permutation Test — ARIMA(2,0,1)
  (Test Set)
==================================================
  Observed Sharpe Ratio : +1.0107
  Permutation p-value   : 0.3433
  Reject H0 at 5%?      : NO
==================================================
```

Conclusion

Key Takeaways

1. **Feature engineering matters:** Adding 10 new features (5 price-based, 5 non-price) improved model diversity. ROC was the single most predictive feature per SelectKBest (score: 7.23), followed by MACD (score: 6.01). Both were from the newly added features, confirming the value of the extensions.
2. **ARIMA(2,0,1) is the best active strategy:** It is the only model with a positive Sharpe ratio and profit factor > 1 on the test set. LASSO overfits to the training bull run and deteriorates sharply out-of-sample.
3. **Beating buy & hold is hard in a bull market:** The 2018-2019 test period was an unusually strong MSFT uptrend. Any strategy that sits flat misses persistent gains, inflating the passive benchmark's advantage.
4. **Statistical validation reveals limits:** The White Reality Check (p=0.3433) and Monte Carlo Permutation Test (p=0.3433) both fail to reject the null. The strategy's test performance is statistically indistinguishable from chance, highlighting the importance of out-of-sample validation beyond simple metrics.
5. **Multicollinearity in SHAP:** SP500, GOOGL and VIXCLS dominate feature importance with mean SHAP values of 0.0043, 0.0031 and 0.0030. Despite SP500 and DJIA being highly correlated market indices, they carry opposing coefficient signs, a classic symptom of multicollinearity inflating and distorting individual coefficients. Future work could apply PCA or drop one index to produce more stable and interpretable feature weights.